Big Data Management and Analytics

Big Data Architectures

P2: Trusted zone, Exploitation zone and Downstream tasks

1. Stages (P2)

1.1 Trusted zone

Raw data comes with all sorts of mistakes or inconsistencies that need to be addressed before it can be used in downstream tasks. This *pre*processing is an essential step of any pipeline that handles data in some capacity. Nonetheless, in the context of our project we are working with a lot of data, and it is very likely that some of this data is going to be used by different exploitation tasks. Hence, executing *generalized* transformations that prepare data for a **specific** application (e.g. training a given ML model) is not a recommended practice, and should be left for further down the pipeline.

For example, a very common preprocessing task prior to training a model is to remove outliers. From the perspective of achieving good predictive performance, removing outliers makes sense, as highly abnormal values can significantly bias the predictions and reduce the efficacy of the average inference. However, if in our pipeline we use the same data to train such a model and also to do, for instance, anomaly detection, the latter task is being significantly conditioned by the processing we did for the former. That is, we are removing the very same thing we aim to detect. Hence, specific data processing techniques should be left for each of the exploitation tasks and their requirements.

As a result, in the trusted zone the goal is to conduct ***generic* data quality tasks** that do not interfere with posterior analytical needs. Another way to think about this is as "correcting the mistakes" that the data contains as a result of its unprocessed ingestion in the landing.

Cleaning the data

Note that the specific processing tasks to be conducted depend on the data that is being handled. For example, structured data such as CSV files can undergo a lot of transformations to ensure its quality, given the constraints associated with the tabular format and a fixed schema. On the other hand, semi-structured data, such as JSON files, might be trickier to process given their variable schema.

We now list some examples of tasks that can be conducted in this zone. Note that not all tasks need to be implemented (nor it might make sense to do so), as these depend on the data that you have:

- Deduplication: removing repeated elements (e.g. based on primary key)
- Schema correction: ensuring data is casted to its proper type.
- Standardization (e.g. define a common format for dates, perform currency conversion or translate text to a specific language).
- Constraint validation, based on business rules (e.g. non-negative ages or correctly formatted emails).
- Schema flattening (e.g., explode arrays, extract nested fields).
- Text normalization (e.g. lowercasing, removing punctuation).
- Spelling correction.

- Compress data for easier handling (mainly for unstructured data).
- Check file integrity and remove/fix corrupted files (mainly for unstructured data).

Tasks

To implement the trusted zone **you need to**:

1. Select the required data storage tools (i.e. databases, see section 2) to implement the trusted zone and set them up.
2. Define where each of your data assets will be stored, and which transformations will be applied to each.
3. Implement the pipelines that move the data from the landing to the exploitation zone. Do so with appropriate technology (see section 2).

1.2 Exploitation zone

In the trusted zone we have cleaned the data, which means that it is ready to be exploited to generate insights. However, this data is likely to be poorly organized, as the bucket/folder distribution assigned in the landing zone is the only structuring performed so far. Hence, our goal in the exploitation zone is to provide the data in the best way possible for analysts to conduct their work.

In doing so, we face again the problem of having to provide data for a potentially large series of analytical tasks. Hence, the data organization plan we define has to obey, not to the needs of a specific downstream system, but to a broader principle of flexibility and reusability. This means that, rather than tailoring datasets for individual reports or models, we aim to create **subject/domain-oriented structures** that can serve multiple purposes. The goal is to empower analysts and data scientists to focus on extracting value from data rather than spending time wrangling or interpreting it.

To achieve this, data is (i) modeled in a way that reflects business entities and their relationships. Additionally, (ii) we might compute new data or structures (described below) that provide additional information to the analyst.

Organizing

The organization of the exploitation zone highly depends on the data that you have, and there is no clear pattern to do so. This lack of a one-size-fits-all approach emphasizes the importance of adapting the structure based on the specific nature of your data. It is useful to think about it as the **ground truth** for subsequent tasks, that is, the data that will be given to the analysts for them to perform whatever processes they need.

The first differentiation is regarding the type of data used. Unstructured data has no schema, so all we can do to facilitate the analyst's work when selecting the data to work with is to store it in buckets that clearly indicate the data that is contained. The same structure employed in the landing zone might suffice, or some, more specific, buckets might need to be created.

Structured and semi-structured data, on the other hand, have a schema, so the organization can be both at the file level as well as at the schema level. This can be more clearly seen with structured data. As we know the schemas beforehand we can perform

joins (i.e. **integrate data**) between tables that provide information about the same set of entities (e.g. patients; we can join a table containing patient visit records with a table containing diagnostic results). On the other hand, we might want to **vertically partition** a table to reduce redundancy. For example, you might have a table where each row identifies an actor appearing in a movie, alongside properties of the actor (age, place of birth, etc.). Due to an actor being able to participate in a lot of movies, you are going to have a lot of repeated values, as the actor information will be replicated for each new entry. As such, you can create a table *Movies*, a table *Actors* and an intermediary table with foreign keys that references both of them.

Similarly, different versions of the same dataset belonging to different timeframes and ingested at different times (e.g. country economic data; one dataset for each month, each with the same set of attributes) can be combined (i.e. **table union**) into a single table that contains all the historical records of that dataset. On the other hand, a table can be **partitioned horizontally** (e.g. based on a criterion such as geographic region) to provide smaller datasets tailored for more specific analyses, which can help optimize storage and performance by grouping similar records together.

Structured data's consistent schema makes these transformations and integrations relatively straightforward. With semi-structured data, such as JSON or XML files, the process is somewhat more complex due to the flexible and potentially nested structure of the data. However, since there is still an implicit schema present, some level of transformation and organization is possible. For instance, fields that appear consistently across entries can be extracted and normalized into structured tables, enabling similar operations like joins or partitions to be performed once the semi-structured data has been partially structured.

Computing new data

Besides organizing the data, we can use the exploitation zone to calculate additional assets that provide extra benefits for the analysts. Once again, the idea is to generate information that can potentially be used by a wide variety of tasks, preventing the need to re-compute these values every time the analytical process needs to be conducted, thereby improving efficiency and consistency across the subsequent tasks.

When working with unstructured data we can enhance its analytical value by extracting structured representations that semantically describe its content. For example, textual data can be summarized, classified into categories (e.g., "review", "complaint", "inquiry"), or converted into embeddings that capture semantic meaning. Similarly, image data can be categorized (e.g., "cat", "dog", "vehicle") or transformed into vector representations for machine learning applications. By storing these semantic characterizations in the exploitation zone, we eliminate the need to repeat these computationally expensive operations during each analysis cycle. This is particularly critical for embedding generation, which is a prerequisite for many machine learning models to interpret and process unstructured inputs.

For semi-structured and structured data, the exploitation zone can also serve as a staging ground for calculating higher-level metrics and aggregates. Leveraging the existing schema, we can compute and store key performance indicators (KPIs) and other summary statistics that are relevant to the organization's strategic goals. These might

include metrics such as average sales per quarter, churn rates, or user engagement trends. By maintaining a historical record of these aggregates, analysts gain immediate access to essential insights that reflect the organization's performance over time, without needing to recompute them with each analysis.

Tasks

To implement the exploitation zone you need to:

1. Define the structure of the exploitation zone, that is, determine how the data will be organized and which transformations you will need to apply.
2. Decide which additional data assets you are going to generate (e.g. KPIs, embedding, labels, etc.) and how.
3. Select the required data storage tools (i.e. databases, see section 2) and set them up
4. Implement the pipelines that move the data from the trusted to the exploitation zone, applying the corresponding transformations. Do so with appropriate technology (see section 2).

We acknowledge that the exploitation zone can be a bit confusing, and the data that you have might severely limit what you can do. Hence, we are not asking you to completely change the structure of the data or to compute *a lot* of new assets. The goal of implementing this zone is, simply, that you play around with the problem of organizing the data in interesting ways to maximize its utility. As long as you provide some coherent explanation for the choices you make, it will be fine.

1.3 Data consumption and exploitation

Note: ***This part is out of the scope of the session****, but we provide it for completeness and reference when developing the final project.*

At this point most of the data management aspects have been fulfilled, and data is ready to be employed in subsequent tasks or fed to external systems. As mentioned in previous sections, **the focus on this course lies in the management aspects rather than on the analytical processes** and, hence, the quality or thoroughness of the steps implemented here are not the main focus on the project.

The priorities for this stage are that you think about how to exploit your data (you should have a clear idea of this at the beginning of the project), that the systems to do so are set up (i.e. you have a place to move the data to), and that the adequate connections between the exploitation zone and the data consumption mechanisms are in place (i.e. something gets in and, ideally, something gets out). For example, if you decide to train an ML model with a subset of the resulting data, we will check whether the data is properly shipped to the appropriate platform, some training process is conducted and the resulting model is stored somewhere. The thoroughness of the training process or the quality of the final model is not a priority.

Some examples of tasks you can implement in this step are:

- Training a machine learning model: Use the processed data to train a predictive model. This could be for classification, regression, clustering, or any other ML task.

- Building a dashboard: Create a visualization tool, such as a Power BI, Tableau, or web-based dashboard, to interactively display key insights from the data.
- Setting up an alert system: Implement a real-time monitoring system that triggers alerts based on specific conditions in the data.
- Developing a recommendation system: Use the data to create personalized recommendations for users.
- Powering a search engine or query system: Make the data searchable through an optimized query system, enabling users to find relevant information quickly.
- Natural language processing (NLP) Tasks: process and analyze textual data using NLP techniques like sentiment analysis or named entity recognition.
- Building a chatbot: create a conversational agent that can understand user queries and provide responses based on the data it is trained on.
- Building an image recognition system: create a system that can recognize objects, faces, or text within images using deep learning techniques.

2. Implementing the zones

2.1 Design

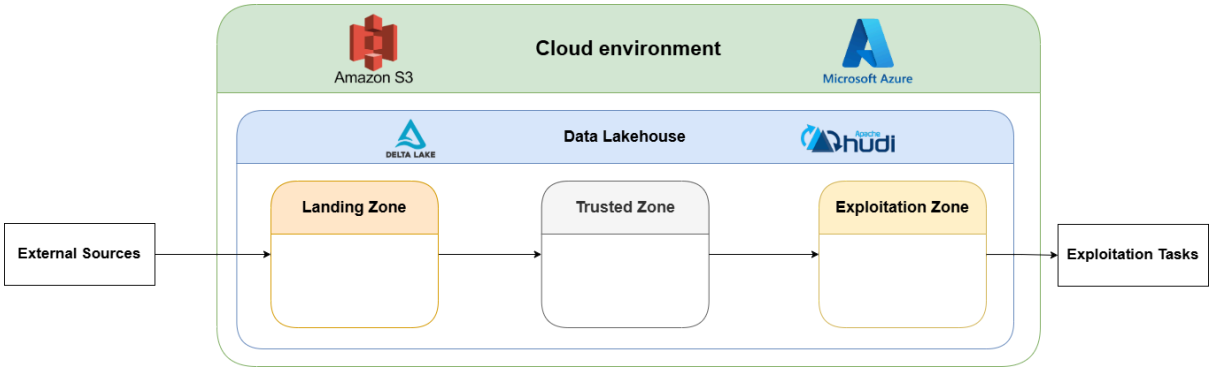
The most important part is that each zone **has to store the data after the respective transformations have been applied**. That is, you have to define separate databases for each zone. For example, in the trusted zone you get the data from the landing zone, apply the necessary cleaning tasks, and store the curated data in a separate database. Similarly for the exploitation zone. In this way we create checkpoints that we can go back to access the data in different stages of maturity, either because of specific analytical needs or due to problems in the execution of subsequent tasks (i.e. we do not have to start all the way from the beginning).

We present two main ways of proceeding with the implementation.

1st option: extending your current architecture

To implement the landing zone, distributed technologies are a must due to the variety and volume of the data that will be handled. Companies often opt for, rather than deploying their own distributed cluster, employing cloud services that handle the infrastructure aspects in a transparent manner. For example, the services of AWS S3 or Microsoft Azure can be acquired, and the company only has to worry about sending the data to their servers and retrieving it when needed. Additionally, table formats (e.g. Delta Lake, Apache Hudi) can be employed “on top” of these distributed systems to provide structured data management and ACID transactions, defining a data lakehouse framework.

In your own project, the landing zone should ideally follow a similar pattern. Depending on your setup, you might have implemented a lakehouse structure on a local file system, or you may be using a cloud environment, or even a combination of both. Regardless of the configuration, **this existing foundation can be extended to implement the remaining zones in your pipeline**. Take the following diagram as an example, in which we combine cloud storage with a data lakehouse architecture:

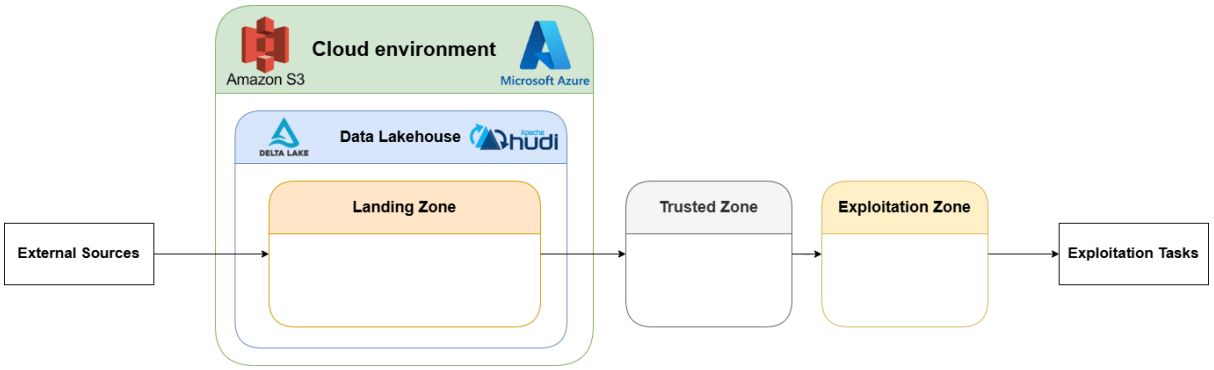


The idea is to take your current system and include more folders/buckets to represent the next zones. The advantage here is that you continue working within an ecosystem you are already familiar with, which reduces the learning curve and simplifies integration. It's important to note that a full cloud-based lakehouse setup is not a strict requirement. If your project already uses a lakehouse on your local file system, you can simply create additional directories to represent the other zones. Similarly, if you're using a cloud provider, adding more buckets or containers to define new zones will suffice.

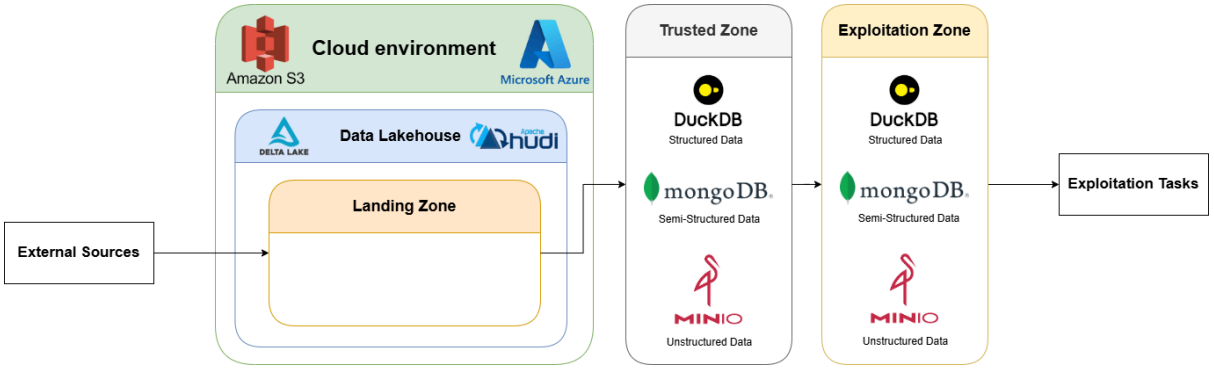
2nd option: moving to smaller, more specialized databases

In traditional data management, before even distributed processing, the pipelines were centered around **data warehouses**. However, these stores could end up containing a lot of data, which could slow the query response time (even in these dedicated systems) and complicate data governance. Moreover, the data ingested by the system might belong to widely different domains, so having all mixed up in a single environment was not ideal.

As a response to that, designers came up with the concept of ***data marts***, subsets of the data warehouse that focuses on a single business function or domain. These smaller repositories are streamlined for quicker querying and a more straightforward setup, catering to the specialized needs of a particular team or function. Their physical implementation was not necessarily different from that of data warehouses, but we can employ this idea of smaller, separated and more specific databases to design a different structure:

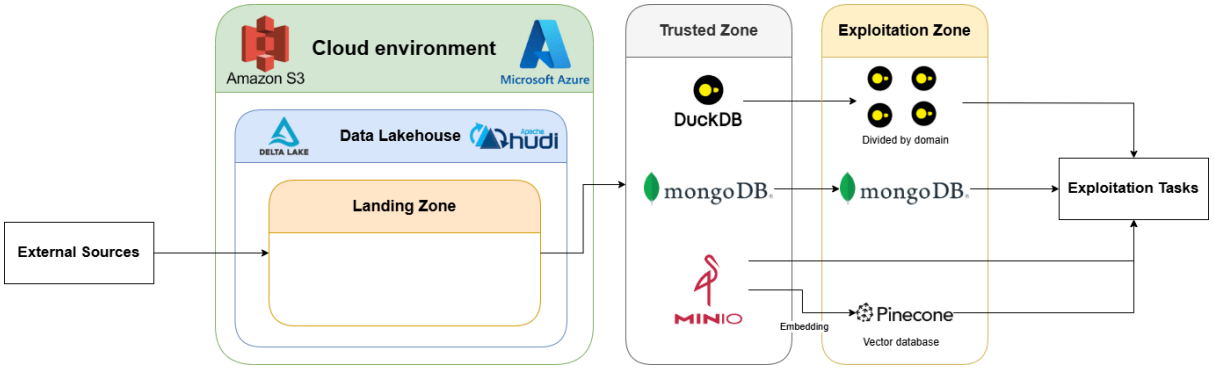


Our new framework is, hence, based on implementing the trusted and exploitation zones in **separate systems**, outside of your chosen technology for the landing zone. This division can be, for example, based on the format of the data:



In the above image we use DuckDB, MongoDB and MinIO to handle each type of data accordingly. These are just examples of tools that you can use, and in the following pages you can find further explanations of each, along with more technologies to implement these zones. As of now, it suffices to know that these are databases, that is, that they can hold data.

Following this idea, you can design the structure that you want based on the criteria that you deem fit. This model will allow you to make more complicated structures, which implies more work on the design aspects, and force you to use different technologies, which is great if there are tools that you want to explore to expand your skill set. A more complex example of this approach can be found next:



In this case, the relational data is separated into several physical stores in the exploitation zone depending on the domain they belong to, which tries to approximate the aforementioned concept of data marts. Additionally, part of the unstructured is transformed into embeddings and stored in a vector database (Pinecone) for efficient handling. The rest of the unstructured data is accessed directly by the exploitation tasks.

Tasks

You need to:

1. Continuing with the work you did with part 1, you have to complete the architecture **diagram** with the new zones, tools, operations, etc.

2. Be sure to adequately reason about the general structure you propose and justify the inclusion of each element.

Note that you can find an **example of the full project diagram** at the end of this document.

2.2 Databases

The choice of database depends on the specific type of data and the use case at hand. Traditionally, relational databases dominated the landscape, but with the growing complexity and volume of data, newer database models (i.e. NoSQL) have emerged to better handle diverse requirements. Next, you can find a brief overview of the main typologies and some recommendations. Before implementing the pipeline you should decide what type of databases better suit your needs.

Relational databases: you should be familiar with this model at this point. Tools like Oracle, MySQL and PostgreSQL still lead the charts for the most popular systems to store data. Nonetheless, these databases store data row-wise, which makes them great for dealing with transactions, but not so much to deal with analytical workloads (mainly based on aggregations). To fix this, new and "modern" relational databases have been created, which have columnar storage, seamless integration with other data science tools and optimized for analytical queries. Our recommendation is that you try some of these new tools, particularly **DuckDB**, which has the advantages of being an embedded database (i.e. the *database* is a file that can be interacted with from anywhere) and is very easy to use.

Key-Value stores: a non-relational database where records are stored and retrieved based on a unique key, similar to a hash map or dictionary. This structure allows for quick lookups and is well-suited for caching and other fast-access requirements. The most popular key-value store is **Redis**.

Document stores: a specialized key-value store, where a document is a nested object (we can think of each document as a JSON). Documents are stored in collections and retrieved by key. Document stores are great for semi-structured data like JSON, XML, or BSON, and allow for flexible schema design. **MongoDB** (which you have seen during the course) is the most popular document store, and ideal if you need fast read/write queries.

Wide-column: optimized for storing large amounts of data with high write throughput and low latency, wide-column stores are designed for scalability. These systems use column families instead of rows, allowing them to efficiently store vast amounts of unstructured or semi-structured data. During the course you have seen **HBase**, which can be set up outside of the Hadoop environment and in an embedded mode. Another alternative is **Apache Cassandra**, which is the most popular wide-column store.

Graph databases: explicitly store data with a mathematical graph structure (as a set of nodes and edges). Roughly speaking, graph databases are a good fit when you want to analyze the connectivity between elements. **Neo4J** is a good option, as you have seen/will see it in other courses and it is the most popular graph database.

Time series: a time series is a series of values organized by time. For example, stock prices might move as trades are executed throughout the day, or a weather sensor will take atmospheric temperatures every minute. Any events that are recorded over time are time-series data. A time-series database is optimized for retrieving and statistical processing of time-series data. The most popular tool is **InfluxDB**.

Vector databases: stores and queries embeddings or vector representations of data. These vectors are typically generated using machine learning models like BERT that map data into high-dimensional space. Vector databases are becoming increasingly popular, especially in fields like NLP and search systems that require high-performance similarity search for multi-dimensional vector data. The most popular tool is **Pinecone**.

Object stores: manages data as objects, rather than files or blocks. This is commonly used in cloud computing, and services like Amazon S3 and Azure Blob Storage are popular examples of object stores. They are a good option for unstructured data, which does not fit neatly into the other database types. Besides cloud options, we recommend **MinIO**, which follows the API of Amazon S3, but can be used in a centralized manner.

4. Deliverables

Main deliverable (P2)

The tasks described in this document correspond to the **second part of the project**, which include:

- **Architecture design** (updated version with the new zones, tools, processes, etc.)
- Stage 3: trusted zone
- Stage 4: exploitation zone

To implement each stage follow the appropriate descriptions, specifically regarding the *tasks* subsections.

5. Pipeline design example